

CONTRATO DE INTEGRAÇÃO DA API

Atualização do Contrato — Versão 4.0

Documento	Contrato de Integração da API — Versão 4.0
Responsável	Back-end da Equipe EasyRide g1-Lagarto
Área	Backend / Frontend
Baseado em	Especificação Técnica do Contrato de Integração da API — Versão 3.0
Formato	JSON / REST

1. Objetivo

Este documento explica os ajustes feitos no contrato original da API — o documento que descreve como o Front-end e o Back-end conversam entre si.

A Versão 4.0 mantém o formato de cadastro e consulta de leads já descrito na Versão 3.0, e acrescenta o que mudou depois de uma auditoria de integração real entre Front-end e Back-end: listas paginadas, login exigido em rotas sensíveis (LGPD), dois códigos de erro novos e a normalização do e-mail.

1.1. Por que o contrato mudou desde o início

No começo, o cadastro de um Lead (uma pessoa interessada no produto) era pensado como um formulário simples, com campos soltos como “nome completo”, “e-mail”, “empresa” e “cargo”.

Só que o banco de dados real é mais detalhado: existem tabelas separadas para **Pessoa**, **Pessoa Física**, **Pessoa Jurídica**, **Modelo** (de cadeira) e **Interesse** (o registro do lead em si). Para o cadastro funcionar direito com essas tabelas, o formato de envio dos dados precisou ser adaptado.

O que vem a seguir é o formato que Front-end e Back-end devem seguir.

2. Cadastro de Lead — `POST /api/leads`

Cada cadastro envia um campo chamado `"tipo_pessoa"`, que só pode valer `"FISICA"` ou `"JURIDICA"`. Esse campo diz ao Back-end **qual conjunto de perguntas validar** — se é o formulário de pessoa física ou o de empresa.

Importante: `tipo_pessoa` existe apenas dentro desse envio de dados (o “payload”). Ele não é guardado como uma coluna na tabela de Pessoa do banco. Serve só de instrução de “qual caminho seguir” — depois de validado, esse valor é descartado. O que realmente diferencia uma pessoa física de uma jurídica no banco é a existência de um registro relacionado na tabela `PessoaFisica` ou na tabela `PessoaJuridica`. Por isso esse campo não aparece no DER (Diagrama de Entidade-Relacionamento): ele não é dado persistido, é só um sinalizador de formulário.

2.1. Envio — Pessoa Física

```
{
  "tipo_pessoa": "FISICA",
  "nome": "João da Silva",
  "email": "joao.silva@email.com",
  "telefone": "11999999999",
  "estado": "SP",
  "cidade": "São Paulo",
  "dados_fisica": {
    "cpf": "123.456.789-00",
    "data_nascimento": "1990-05-20",
    "tipo_deficiencia": "Motora",
    "perfil": "paciente",
    "comunicacao_verbal_preservada": true
  },
  "interesse": {
    "modelo_id": 2,
    "quantidade_estimada": 1,
    "mensagem": "Gostaria de mais detalhes sobre as dimensões da cadeira.",
    "origem": "outro",
    "aceite_termos": true,
    "possui_cadeira": false
  }
}
```

Valores aceitos (nada além disso passa na validação):

- `dados_fisica.perfil`: `"paciente"`, `"familiar"` ou `"cuidador"`.
- `interesse.origem`: `"google"`, `"instagram"`, `"facebook"`, `"indicacao"`, `"evento"` ou `"outro"`.
- O nome correto do campo é `comunicacao_verbal_preservada` (no feminino).

2.2. Envio — Pessoa Jurídica

```
{
  "tipo_pessoa": "JURIDICA",
  "nome": "Clínica de Reabilitação Esperança",
  "email": "contato@clinicaesperanca.com.br",
  "telefone": "1132221111",
  "estado": "SP",
  "cidade": "Campinas",
  "dados_juridica": {
    "cnpj": "12.345.678/0001-90",
    "tipo_instituicao": "clinica",
    "contato_responsavel": "Maria Souza",
    "cargo_responsavel": "Diretora Médica"
  },
  "interesse": {
    "modelo_id": 5,
    "quantidade_estimada": 10,
    "mensagem": "Cotação para renovação da frota da clínica.",
    "origem": "google",
    "aceite_termos": true,
    "possui_cadeira": true
  }
}
```

Valor aceito:

- `dados_juridica.tipo_instituicao`: `"clinica"`, `"hospital"`, `"ong"` ou `"outro"`.

3. Resposta de sucesso — 201 Created

Quando o cadastro dá certo, a API responde assim:

```
{
  "status": "success",
  "mensagem": "Lead cadastrado com sucesso",
  "dados": {
    "id": 1,
    "tipo_pessoa": "FISICA",
    "nome": "João da Silva",
    "email": "joao.silva@email.com"
  }
}
```

4. Listagem de Leads — `GET /api/leads` e `GET /api/leads/{id}`

Nessa listagem, quem organiza os dados é o **Interesse** (o lead em si). Cada item já vem com os dados da **Pessoa** (incluindo se é física ou jurídica) e do **Modelo** de cadeira escolhido, para o Front-end não precisar fazer uma segunda consulta.

```
[
  {
    "id": 15,
    "data_hora": "2026-06-25T14:30:00Z",
    "status_lead": "novo",
    "quantidade_estimada": 1,
    "mensagem": "Gostaria de mais detalhes sobre as dimensões da cadeira.",
    "origem": "outro",
    "possui_cadeira": false,
    "aceite_termos": true,
    "pessoa": {
      "id": 10,
      "tipo_pessoa": "FISICA",
      "nome": "João da Silva",
      "email": "joao.silva@email.com",
      "telefone": "11999999999",
      "estado": "SP",
      "cidade": "São Paulo",
      "dados_fisica": {
        "cpf": "123.456.789-00",
        "data_nascimento": "1990-05-20",
        "tipo_deficiencia": "Motora",
        "perfil": "paciente",
        "comunicacao_verbal_preservada": true
      }
    },
    "modelo": {
      "id": 2,
      "nome_modelo": "Cadeira Motorizada X",
      "marca": "Marca Y",
      "motorizada": true
    }
  }
]
```

Nota: aqui dentro de `pessoa`, o campo `tipo_pessoa` é diferente do que foi explicado na seção 2 — este é **calculado na hora da resposta**, olhando se existe `dados_fisica` ou `dados_juridica` junto. Também não é uma coluna do banco; é só um texto pronto para o Front-end não precisar descobrir sozinho qual tipo de pessoa é aquela.

5. Respostas de erro

Erro de validação — 400 Bad Request

Quando algum campo está errado ou faltando, a API aponta exatamente onde:

```
{
  "email": [
    "Este campo é obrigatório."
  ],
  "dados_fisica": {
    "cpf": [
      "Formato de CPF inválido."
    ]
  }
}
```

Conflito (e-mail já cadastrado) — 409 Conflict

```
{
  "status": "error",
  "mensagem": "Lead já cadastrado"
}
```

Não encontrado — 404 Not Found

```
{
  "status": "error",
  "mensagem": "Lead não encontrado"
}
```

```
{
  "status": "error",
  "mensagem": "Erro interno no servidor"
}
```

O que a Versão 4.0 acrescenta (Entrega 3, julho de 2026)

Esta parte explica comportamentos da API que foram descobertos e ajustados durante uma auditoria de integração entre Front-end e Back-end, feita em 24/07/2026 (relatório completo em [testes/relatorio/AUDITORIA_DIVERGENCIAS.md](#)).

6. Listas vêm “embrulhadas”, não soltas

Os exemplos acima mostram listas como um array simples (`[ {...}, {...}]`). Na prática, toda lista da API vem dentro de um “envelope” com informação de paginação:

```
{
  "count": 42,
  "next": "http://.../api/leads/?page=2",
  "previous": null,
  "results": [ { "...": "os itens, no formato mostrado acima" } ]
}
```

Isso vale para `GET /api/leads/`, `/api/pessoas/`, `/api/modelos/`, `/api/beneficios/`, `/api/depoimentos/` e `/api/faq/`. Isso diverge do que a Versão 3.0 descrevia (array JSON puro nesses mesmos endpoints) — é uma correção de documentação, não uma mudança de comportamento: o Back-end já paginava essas rotas antes mesmo da Versão 4.0, só não estava documentado corretamente. Quem for consumir a API deve pegar a lista de dentro de `results`, e não tratar a resposta inteira como se já fosse a lista.

7. Algumas rotas de leitura pedem login

As rotas abaixo mostram dados sensíveis da pessoa (CPF, CNPJ, telefone e o tipo de deficiência, que é informação de saúde protegida por lei). Essa exigência de login existe para manter o sistema em conformidade com a LGPD (Lei Geral de Proteção de Dados) — dados pessoais e de saúde não podem ficar abertos para qualquer visitante. Por isso, só um usuário da equipe (staff) logado consegue acessá-las:

Rota	Sem login
GET /api/leads/	401 Unauthorized
GET /api/leads/{id}/	401 Unauthorized
GET /api/pessoas/	401 Unauthorized
GET /api/leads/estatisticas/	401 Unauthorized

O `POST /api/leads/` (o cadastro em si, feito pelo site) continua público, como já era combinado — só a leitura dos dados já cadastrados é que exige login.

8. Dois códigos de erro a mais

Situação	Código	O que a API devolve
A requisição não diz que está enviando JSON	415	<code>{"detail": "..."} </code>
Mais de 5 cadastros de lead por minuto, vindos do mesmo lugar	429	<code>{"detail": "..."} </code>

Duas observações sobre isso:

- O `415` acontece **antes** de qualquer validação de campo — é um erro de formato de envio, não de conteúdo. Por isso o Front-end sempre deve avisar no envio que está mandando JSON.
- O `429` é uma proteção contra spam no formulário público: mais de 5 envios por minuto do mesmo lugar são bloqueados por um tempo.

9. E-mail não diferencia maiúscula de minúscula

Se alguém cadastra `Ana@Email.com` e depois `ana@email.com`, o sistema trata como o **mesmo** cadastro — o segundo envio recebe o erro de “já cadastrado” (`409`). O e-mail é sempre guardado em letras minúsculas, não importa como foi digitado.

10. Histórico de Versões

Versão	Data	O que mudou
2.0	Entrega 2	Formato inicial “achatado” evoluído para serializers aninhados (<code>tipo_pessoa</code> , <code>dados_fisica</code> / <code>dados_juridica</code> , <code>interesse</code>).
3.0	Julho de 2026	Consolidação dos serializers aninhados e criação das rotas de conteúdo dinâmico da Landing Page (<code>/api/beneficios</code> , <code>/api/depoimentos</code> , <code>/api/faq</code>).
4.0	Julho de 2026 (Entrega 3)	Documento atual. Paginação confirmada em todas as listagens, login obrigatório por LGPD nas rotas com dados sensíveis, códigos <code>415</code> e <code>429</code> documentados, e-mail normalizado para minúsculas.